

Manuel d'Exploitation & Runbook Opérationnel – `einvoice-saas` (v1.0 Production)

Statut du Document : Référence d'exploitation officielle v1.0

Champ d'application : Production, MCO, Déploiement continu, Supervision fiscale & EDI

Système d'exploitation cible : Windows Server / Windows 10/11 (PowerShell 5.1 & PowerShell 7 Core)

Objectifs de service (SLA) :

- **RPO (Recovery Point Objective)** : \$le 15\text{ minutes}\$ (instantanés SQLite réguliers + WAL actif).
- **RTO (Recovery Time Objective)** : \$le 30\text{ minutes}\$ (reprise sur binaire sec et restauration de backup intègre).

1. Matrice RACI des Opérations Critiques

Toute action sur l'environnement de production doit respecter la séparation stricte des responsabilités ci-dessous :

Opération	**Dev (Ingénierie)**	**Ops / MCO**	**Pilotage des Flux**	**R&D / Conformité**
Release & Déploiement	**R** esponsable (Prépare)	**A** pprobateur / Exécute	**C** onsulté	**A** pprobateur (Certifie)
Rollback Applicatif / DB	**C** onsulté	**A** pprobateur / Exécute	**I** nformé	**A** pprobateur
Replay Outbox FAILED	**I** nformé	**A** pprobateur / Exécute	**R** esponsable (Valide le lot)	**I** nformé
Restauration DB d'Urgence	**C** onsulté	**A** pprobateur / Exécute	**R** esponsable (Valide la cohérence)	**I** nformé
Purge de Brouillons / Rétention	**I** nformé	**A** pprobateur / Exécute	**R** esponsable (Valide la plage)	**I** nformé
Qualification Factur-X / SVRL	**R** esponsable (Corrige)	**I** nformé	**C** onsulté	**A** pprobateur (Valide veraPDF)

*Légende : **R** = Réalise (Responsable), **A** = Approuve & Décide (Accountable), **C** = Consulté, **I** = Informé.*

2. Gestion Sécurisée des Secrets & Contrôles d'Accès

Règle absolue : Aucun secret (jeton JWT, identifiant PDP, certificat PISTE, mot de passe administrateur) ne doit être injecté en clair dans le code, dans les scripts .ps1, ni dans l'historique de la console.

Procédure de saisie sécurisée en mémoire volatile (Console PowerShell)

PowerShell

```
# 1. Capture interactive sans echo console
$secureToken = Read-Host -Prompt "Entrez le JWT Administrateur" -AsSecureString
$bst = [System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($secureToken)
$adminToken = [System.Runtime.InteropServices.Marshal]::PtrToStringAuto($bst)
[System.Runtime.InteropServices.Marshal]::ZeroFreeBSTR($bst)

# 2. Construction de l'en-tête d'authentification
$authHeaders = @{
    "Authorization" = "Bearer $adminToken"
    "Content-Type" = "application/json"
}

# 3. Purgatoire : Effacement immédiat de la variable en mémoire après utilisation
function Clear-AdminSecret {
    $global:adminToken = $null
    Remove-Variable -Name adminToken -Scope Global -ErrorAction SilentlyContinue
    [System.GC]::Collect()
    Write-Host "[SECURITE] Jeton administrateur purge de la memoire." -ForegroundColor Yellow
}
```

3. Pré-requis, Pre-Flight Checks & Critères Go / No-Go

Avant tout déploiement ou action MCO, l'ingénieur d'astreinte exécute le script de vérification ci-dessous. Le résultat doit afficher GO sur tous les critères.

PowerShell

```
Set-Location -LiteralPath "D:\einvoice-saas-release1\einvoice-saas"

Write-Host "=== PRE-FLIGHT CHECK DE PRODUCTION ===" -ForegroundColor Cyan
$goStatus = $true

# A. Verification de l'espace disque (Seuil minimum : 15 Go libres pour WAL + PDF)
$drive = Get-PSDrive -Name D
$freeGB = [math]::Round($drive.Free / 1GB, 2)
if ($freeGB -lt 15) {
    Write-Warning "[NO-GO] Espace disque D: insuffisant ($freeGB Go restants, 15 Go requis)"
    $goStatus = $false
} else {
    Write-Host "[OK] Espace disque suffisant ($freeGB Go libres)" -ForegroundColor Green
}

# B. Verification des binaires requis
@("go", "sqlite3", "git") | ForEach-Object {
    if (-not (Get-Command $_ -ErrorAction SilentlyContinue)) {
        Write-Warning "[NO-GO] Outil systeme manquant : $_"
        $goStatus = $false
    } else {
        Write-Host "[OK] Outil disponible : $_" -ForegroundColor Green
    }
}

# C. Verification de l'arbre Git
$branch = (git rev-parse --abbrev-ref HEAD).Trim()
$uncommitted = (git status --porcelain).Length
if ($uncommitted -ne 0) {
    Write-Warning "[NO-GO] Arbre Git non propre ($uncommitted fichiers non commites)"
    $goStatus = $false
} else {
    Write-Host "[OK] Arbre Git propre sur la branche '$branch'" -ForegroundColor Green
}

# Verdict
if ($goStatus) {
    Write-Host "-> STATUT PRE-FLIGHT : [GO DEPLOIEMENT]" -ForegroundColor Green
} else {
    Write-Error "-> STATUT PRE-FLIGHT : [NO-GO DETECTE] Interdiction de deployer."
}
```

4. Sauvegarde SQLite Industrielle, Intégrité & Restauration

La sauvegarde de production ne s'arrête pas à VACUUM INTO. Elle comprend le contrôle de cohérence PRAGMA, la validation du hash SHA-256 et la rotation rétentionnelle.

A. Procédure de Sauvegarde à Chaud avec Contrôle d'Intégrité

PowerShell

```
$ErrorActionPreference = "Stop"
$backupDir = "D:\einvoice-saas-release1\backups\db"
if (!(Test-Path $backupDir)) { New-Item -ItemType Directory -Path $backupDir -Force }

$timestamp = (Get-Date).ToString("yyyyMMdd_HH:mm:ss")
$dbPath = "D:\einvoice-saas-release1\einvoice-saas\data\invoices.db"
$backupTarget = "$backupDir\invoices_$timestamp.db"

Write-Host "1. Creation du snapshot atomique SQLite..." -ForegroundColor Cyan
sqlite3 $dbPath "VACUUM INTO '$backupTarget';"

Write-Host "2. Verification d'integrite du fichier de sauvegarde..." -ForegroundColor Cyan
$checkResult = (sqlite3 $backupTarget "PRAGMA quick_check;").Trim()
if ($checkResult -ne "ok") {
    Remove-Item $backupTarget -Force
    Write-Error "[ALERTE PRODUCTION] Sauvegarde corrompue ($checkResult) ! Fichier elimine."
}

# 3. Calcul de l'empreinte SHA-256
$hash = (Get-FileHash -Path $backupTarget -Algorithm SHA256).Hash
Set-Content -Path "$backupTarget.sha256" -Value "$hash invoices_$timestamp.db"
```

```
Write-Host "[OK] Sauvegarde validee (SHA256: $hash)" -ForegroundColor Green

# 4. Rotation : Conservation stricte des 30 derniers snapshots (politique MCO)
Get-ChildItem -Path $backupDir -Filter "invoices_*.db" |
Sort-Object CreationTime -Descending |
Select-Object -Skip 30 |
ForEach-Object {
    Remove-Item $_.FullName -Force
    Remove-Item "$($_.FullName).sha256" -Force -ErrorAction SilentlyContinue
    Write-Host "Purge ancienne sauvegarde : $($_.Name)" -ForegroundColor DarkGray
}
```

B. Procédure de Restauration d'Urgence (Crash Recovery)

En cas de corruption de la base active ou de demande de retour arrière :

PowerShell

```
param(
    [Parameter(Mandatory=$true)]
    [string]$SnapshotPath
)

# 1. Arrêt immédiat de tout processus accédant à la base
$conn = Get-NetTCPConnection -LocalPort 8080 -State Listen -ErrorAction SilentlyContinue
if ($conn) {
    Stop-Process -Id $conn.OwningProcess -Force
    Write-Host "[OPS] Serveur arrêté pour restauration de base." -ForegroundColor Yellow
}

# 2. Vérification d'intégrité de l'empreinte SHA-256
$recordedHash = (Get-Content "$SnapshotPath.sha256").Split(" ")[0].Trim()
$computedHash = (Get-FileHash -Path $SnapshotPath -Algorithm SHA256).Hash
if ($recordedHash -ne $computedHash) {
    Write-Error "[ALERTE] Le hash du snapshot ne correspond pas ! Restauration refusée."
}

# 3. Mise en quarantaine de la base endommagée
$dbActive = "D:\invoice-saas-release1\invoice-saas\data\invoices.db"
$quarantineTarget = "D:\invoice-saas-release1\backups\quarantine_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').db"
Move-Item -Path $dbActive -Destination $quarantineTarget -Force
Remove-Item "$dbActive-wal" -Force -ErrorAction SilentlyContinue
Remove-Item "$dbActive-shm" -Force -ErrorAction SilentlyContinue

# 4. Restauration de la sauvegarde validee
Copy-Item -Path $SnapshotPath -Destination $dbActive -Force
Write-Host "-> [SUCCES] Base restaurée depuis le snapshot $SnapshotPath" -ForegroundColor Green
```

5. Déploiement Sécurisé & Procédure de Rollback

A. Déploiement Applicatif (Binaire compilé versionné)

On bannit `go run` en production au profit d'un binaire figé, tagué et versionné.

PowerShell

```
Set-Location -LiteralPath "D:\invoice-saas-release1\invoice-saas"

$targetVersion = "v0.2.0"
Write-Host "1. Compilation du binaire de release $targetVersion..." -ForegroundColor Cyan
$env:CGO_ENABLED = "0"
go build -ldflags "-s -w -X main.Version=$targetVersion" -o "bin\invoice-server-$targetVersion.exe" ./cmd/server

# 2. Permutation à chaud
$conn = Get-NetTCPConnection -LocalPort 8080 -State Listen -ErrorAction SilentlyContinue
if ($conn) {
    Stop-Process -Id $conn.OwningProcess -Force
    Start-Sleep -Seconds 1
}

# 3. Démarrage supervisé avec polling dynamique de disponibilité
Start-Process -FilePath "bin\invoice-server-$targetVersion.exe" -WindowStyle Hidden

$ready = $false
$maxRetries = 20
$attempt = 0
while (-not $ready -and $attempt -lt $maxRetries) {
    $attempt++
    Start-Sleep -Milliseconds 500
}
```

```

try {
    $resp = Invoke-RestMethod -Uri "http://localhost:8080/health" -TimeoutSec 2 -ErrorAction Stop
    if ($resp.status -eq "ok" -and $resp.database -eq "connected") {
        $ready = $true
    }
} catch {}

if (-not $ready) {
    Write-Error "[ECHEC CRITIQUE] Le serveur n'a pas valide son readiness apres $(($maxRetries * 0.5) secondes !"
    Write-Warning "Enclenchement automatique de la procedure de ROLLBACK..."
    # Appel de la procedure de rollback (Section 5.B)
} else {
    Write-Host "[OK] Serveur démarre et operationnel sous $targetVersion." -ForegroundColor Green
}

```

B. Procédure de Rollback Applicatif & Vercel

En cas d'anomalie post-déploiement :

PowerShell

```

Write-Host "=== ENCLenchement DU ROLLBACK DE PRODUCTION ===" -ForegroundColor Red

# 1. Arrêt du binaire défaillant
$conn = Get-NetTCPConnection -LocalPort 8080 -State Listen -ErrorAction SilentlyContinue
if ($conn) { Stop-Process -Id $conn.OwningProcess -Force }

# 2. Relance immédiate du binaire stable précédent (N-1)
$previousStableVersion = "v0.1.0"
Write-Host "Redémarrage d'urgence sur la version stable $previousStableVersion..." -ForegroundColor Yellow
Start-Process -FilePath "bin\invoice-server-$previousStableVersion.exe" -WindowStyle Hidden

# 3. Rollback du frontend Vercel vers le déploiement précédent
# (Permute le domaine de production vers le dernier alias stable sans rebuild)
npmx vercel rollback --yes

Write-Host "-> [ROLLBACK COMPLETE] Production rétablie sur $previousStableVersion." -ForegroundColor Green

```

6. Pilotage des Flux : Rejeu Sécurisé de l'Outbox Multi-Tenant

Interdiction formelle de pratiquer des UPDATE outbox_events SET status = 'PENDING' globaux.

Tout rejeu doit être **cloisonné par Tenant**, limité en volume, audité et précédé d'une analyse des causes.

Procédure de Rejeu Sécurisé (Scoped Replay)

PowerShell

```

param(
    [Parameter(Mandatory=$true)]
    [string]$TenantId,
    [Parameter(Mandatory=$true)]
    [string]$CauseFilter, # Ex: '%timeout%', '%connection refused%'
    [int]$BatchLimit = 50
)

$dbPath = "data\invoices.db"

# 1. Diagnostic préalable du lot à rejouer
Write-Host "--- ANALYSE DU LOT OUTBOX A REJOUER ($TenantId) ---" -ForegroundColor Cyan
sqlite3 -header -column $dbPath @
SELECT id, topic, retry_count, last_error, updated_at
FROM outbox_events
WHERE tenant_id = '$TenantId'
AND status = 'FAILED'
AND last_error LIKE '$CauseFilter'
LIMIT $BatchLimit;
"@

# 2. Demande de confirmation explicite avec enregistrement de l'opérateur
$confirm = Read-Host -Prompt "Confirmez-vous le rejeu de ces evenements pour le tenant $TenantId ? (OUI/NON)"
if ($confirm -ne "OUI") {
    Write-Warning "Rejeu annulé par l'opérateur."
    return
}

# 3. Rejeu atomique borne avec journalisation dans la table d'audit

```

```

$operator = $env:USERNAME
sqlite3 $dbPath @"
BEGIN TRANSACTION;

-- Inscription dans les logs d'audit systeme
INSERT INTO audit_logs (tenant_id, action, performed_by, details, created_at)
VALUES ('$TenantId', 'OUTBOX_REPLAY', '$operator', 'Rejeu filtre: $CauseFilter, limite: $BatchLimit', datetime('now'));

-- Re-aiguillage des evenements cibles uniquement
UPDATE outbox_events
SET status = 'PENDING',
    retry_count = 0,
    next_retry_at = datetime('now')
WHERE id IN (
    SELECT id FROM outbox_events
    WHERE tenant_id = '$TenantId'
    AND status = 'FAILED'
    AND last_error LIKE '$CauseFilter'
    LIMIT $BatchLimit
);

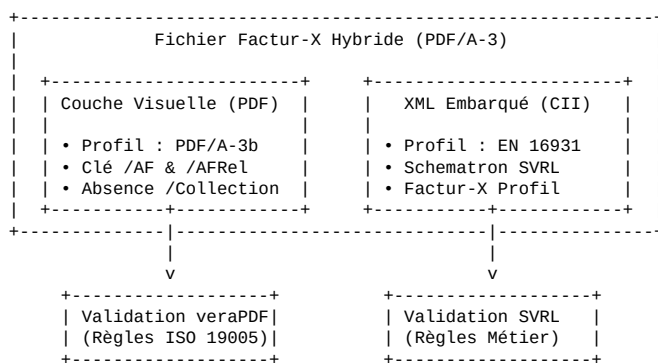
COMMIT;
"@

Write-Host "-> [SUCCES] Lot de $BatchLimit evenements re-enfile pour le tenant $TenantId." -ForegroundColor Green

```

7. R&D & Conformité : Qualification Factur-X & Schematron EN 16931

L'analyse textuelle de motifs /Collection n'est qu'un filtre préalable. La conformité Factur-X exige une double validation : la couche graphique/archivage (PDF/A-3b) et la couche sémantique XML (CII / Schematron EN 16931).



A. Contrôle Structurel d'Embarquement (Pre-check rapide)

PowerShell

```

function Test-FacturXStructure {
    param([string]$PdfPath)

    $bytes = [System.IO.File]::ReadAllBytes($PdfPath)
    $content = [System.Text.Encoding]::ASCII.GetString($bytes)

    $hasCollection = $content.Contains("/Collection")
    $hasAF = $content.Contains("/AF")
    $hasAFRel = $content.Contains("/AFRelationship")

    if ($hasCollection) {
        Write-Warning "[NON-CONFORME] Portfolio PDF interdit (/Collection detecte)."
        return $false
    }
    if (-not $hasAF -or -not $hasAFRel) {
        Write-Warning "[NON-CONFORME] Balises d'association normative /AF manquantes."
        return $false
    }

    Write-Host "[PRE-CHECK OK] Structure du conteneur conforme (veraPDF requis pour validation finale)." -ForegroundColor Green
    return $true
}

```

B. Qualification Finale veraPDF & Schematron (Critère Bloquant Release)

Avant de taguer toute version de production, l'équipe R&D & Conformité doit exécuter :

PowerShell

```
# 1. Validation ISO Schematron / SVRL des regles metier EN 16931
go test -v -run "TestSchematronEngine_Validate|TestSVRLParser" ./pkg/validation ./internal/validator

# 2. Validation formelle PDF/A-3 via veraPDF CLI (en conteneur ou binaire local)
# Doit renvoyer compliant="true"
verapdf --flavour 3b --format text "test/data/facturx_sample.pdf"
```

Critère d'acceptation de release : **Zéro warning SVRL de sévérité fatal**, profil PDF/A-3b certifié compliant=true.

8. Stratégie de Surveillance, Métriques & Alerting Automatisé

L'observabilité repose sur la surveillance périodique des seuils critiques :

PowerShell

```
# Script d'inspection des metriques Outbox pour sonde de monitoring (Zabbix, Datadog ou Prometheus)
$dbPath = "data\invoices.db"

$metrics = sqlite3 -separator "|" $dbPath @"
SELECT
    COUNT(CASE WHEN status = 'FAILED' THEN 1 END) AS failed_count,
    COUNT(CASE WHEN status = 'PENDING' THEN 1 END) AS pending_count,
    COALESCE(MAX(CAST((julianday('now') - julianday(created_at)) * 86400 AS INTEGER)), 0) AS oldest_pending_sec
FROM outbox_events;
"@

$failed, $pending, $oldestSec = $metrics.Split("|")

Write-Host "Métriques : FAILED=$failed | PENDING=$pending | RETARD_MAX=$oldestSec s"

# Évaluation des seuils d'alerte opérationnels
if ([int]$failed -gt 10) {
    Write-Error "[ALERTE SEV-2] Nombre d'échecs Outbox anormalement élevé ($failed événements FAILED) !"
}
if ([int]$oldestSec -gt 900) {
    Write-Error "[ALERTE SEV-1] Blocage de chaîne EDI : un événement est en attente depuis plus de 15 minutes !"
}
```

Grille des Seuils de Supervision

Indicateur	**Seuil Attention (Warning)**	**Seuil Critique (Alerte P1)**	**Action Immédiate**
Outbox FAILED	\$> 0\$	\$> 10\$	Analyse des logs + Scoped Replay (Section 6)
Âge d'un message PENDING	\$> 5\text{ min}\$	\$> 15\text{ min}\$	Redémarrage du worker Outbox
Espace disque D:	\$< 15\text{ Go}\$	\$< 5\text{ Go}\$	Purge des vieux backups & archives temporaires
Sonde `/health`	Timeout \$> 2\text{ s}\$	Non joignable	Déclenchement Crash Recovery / Rollback